STEREO VISION CAPSTONE · BENCHMARK REPORT

23 stereo matching methods, 15 scenes, and what separated them

run **20261105-035941** · when **2026-11-05 04:26:54 UTC** · cells **345 of 345 succeeded** · scenes **15** · algorithms **23** · scored on **the `nonocc` region**

timing **median of 1 run(s), 1 warmup** · threads **library default** · cpu **13th Gen Intel(R) Core(TM) i9-13900HX** · gpu **none (CUDA unavailable)**

Summary · What the numbers mean · How to read the metrics · Leaderboard · Accuracy vs time · What moved the numbers · Exposure robustness · Scene by scene · Compute cost · Looking at the output · What this does not show · Appendix

01 Summary

23 stereo matching methods were run on 15 rectified pairs with ground truth, scored on the `nonocc` region and profiled for time and memory on one machine. The most accurate was `learned_raft_stereo_r4` at **10.5%** of pixels off by more than 2 px; the fastest was `opencv_bm` at **0.026 s** per pair (36.4%). The run's main result is that the **cost function** dominates: with everything else held fixed, swapping the cost function moves the error by 18.7 points against 12.7 for the aggregation strategy. On the pairs that differ only in exposure or lighting, structure-based costs lose +1.3 points where intensity-based costs lose +25.6. Every error figure appears twice — once counting unanswered pixels as errors, once over answered pixels only — because the two orderings are different and a method can be made to look good by answering less.

02 What the numbers mean

Each statement below is derived from this run's data, not written in advance. The supporting measurement is given under each one.

• learned_raft_stereo_r4 is the most accurate method here

HEADLINE

It leaves **10.5%** of evaluated pixels more than 2 px from the truth, averaged over 15 scenes, and takes **4.43 s** per pair. Read that number as an upper bound on quality for this field, not as a good result in absolute terms — one pixel in 10 is still wrong.

mean over the nonocc region, 23 algorithms compared

• A from-scratch matcher is competitive with the libraries

HEADLINE

`scratch_census_sgm4` — a few hundred lines of NumPy — scores **12.4%**, and it lands **1.9 points** behind `learned_raft_stereo_r4`, while beating the best `opencv` entry. Classical stereo is not a black box: the accuracy is in the cost function and the aggregation, both of which fit on a page.

learned: 10.5%, torch: 12.3%, scratch: 12.4%, opencv: 22.7%

• The cost function matters more than the other half of the pipeline

HEADLINE

Holding the pipeline fixed and changing only the **cost function** moves `bad 2px` by **18.7 points** (ssd → census: 36.0% → 17.3%). Changing only the **aggregation** moves it by **12.7** (block matching → semi-global block matching: 36.4% → 23.7%). How you compare two patches is the decision that counts; smoothing a bad cost volume cannot recover what the comparison threw away.

the same family, aggregation and post-processing (scratch / wta); the same family, cost function and post-processing (opencv)

• Only structure-based costs survive an exposure change

HEADLINE

The dataset ships pairs that differ in exposure or lighting but not in geometry, so the extra error is caused by the photometric change and nothing else. Costs that compare *structure* (census, ZNCC) gain **+1.3 points** on average; costs that compare *raw intensity* (SAD, SSD) gain **+25.6**. The worst case is `scratch_sad_wta_lrcheck` at **+53.5**. This is the single clearest mechanism in the whole run: census discards brightness and keeps only the ordering of neighbouring pixels, so a gain change is invisible to it.

8 invariant vs 5 intensity-based methods

⚠ Sparse methods look far better than they are

CAUTION

`opencv_bm` answers only **71%** of pixels. Score it only where it answered and it is **10.5%** wrong, ranking **#2**; count the pixels it skipped as errors — which is what a downstream consumer of a depth map experiences — and it is **36.4%** wrong, ranking **#20**. This is why every error figure in this report is given twice. A method that declines to guess is not thereby accurate.

density 71%, gap 25.9 points between the two framings

⚠ Wall time hides what a method actually costs

CAUTION

`torch_sad_wta` and `scratch_sad_wta` compute the same answer (35.8% and 35.8%) in a comparable wall time (2.19 s and 1.79 s), but the first burns **30 CPU-seconds** to the second's **1.8** — a **16×** difference in energy for the same result. On a laptop sharing cores with anything else, or on a robot with a power budget, wall time is the wrong column.

cpu_s / wall_s is the 'cores busy' column

⚠ learned_raft_stereo_r1 did not converge — do not read it as a result

CAUTION

Its average error is **25.2 px** against a field median of 2.4 px, at 61.2% bad pixels. An error that size on a dense output is not a hard scene, it is a method running outside the conditions it needs — too few refinement iterations, or an input normalisation that does not match what the weights expect. It is kept in the table because removing rows that disappoint you is how benchmarks start lying.

field median EPE 2.4 px

○ The accuracy you can buy with time, and where it stops

DETAIL

7 of 23 methods are not beaten on both axes at once. Moving along that frontier from `opencv_bm` to `learned_raft_stereo_r4` costs **174×** the time and buys **25.9 points** of accuracy. Everything off the frontier is strictly dominated — slower *and* less accurate than something else in the table — and needs a reason other than accuracy or speed to be chosen.

Pareto frontier on (wall time, bad 2px)

○ Averages over scenes hide where the methods differ

DETAIL

PianoL is the hardest scene in the run (mean 44.8%, best 17.4% by `learned_raft_stereo_r4`) and **Motorcycle** the easiest (14.5%). But the scene worth looking at is **Playtable**, where the field spans **78 points** from 6.6% to 84.9%. A method's mean can look respectable while it collapses on one scene type, which is the failure mode that matters if you only ever deploy on that type.

15 scenes, spread measured across all methods

03 How to read the metrics

A stereo result cannot be summarised by one number, because "wrong" has at least three meanings: how often, how far, and how much of the image was answered at all.

Accuracy — was it right?

`bad2_all` **bad 2px** % lower is better

The headline number. For every pixel in the scoring region, is the estimate within 2 px of the truth? A pixel the method declined to estimate counts as wrong, which is the only framing that compares a dense method with a sparse one honestly. 2 px is the Middlebury convention; it is loose enough to ignore sub-pixel noise and tight enough that a surface at the wrong depth fails it.

`bad2_valid` **bad 2px, answered** % lower is better

The same test restricted to pixels the method answered. This measures the quality of the answers it chose to give, which is the right question when a later stage will fill the holes anyway — and the wrong one if nothing will.

`bad1_all` **bad 1px** % lower is better

Tightening the threshold separates methods that get the surface roughly right from methods that get it precisely right. A method with good `bad 2px` and poor `bad 1px` is finding the correct surface but quantising it — usually sub-pixel refinement turned off.

`bad0p5_all` **bad 0.5px** % lower is better

Almost entirely a test of sub-pixel interpolation. Without it, disparities are integers and roughly half of all pixels fail this by construction.

`epe_valid` **EPE** px lower is better

End-point error: the average of |estimate − truth| in pixels. Where `bad N` asks how *often* a method is wrong, EPE asks how *far*. A mean is dragged by outliers, so a method with a low bad-pixel rate and a high EPE is mostly right with a few catastrophically wrong regions — typically a repeated texture matched to the wrong period.

`rmse_valid` **RMSE** px lower is better

Squaring before averaging weights a single 20 px blunder like a hundred 2 px wobbles. Compare it against EPE: a large ratio means the error is concentrated in a few places rather than spread thin.

`a95` **A95** px lower is better

A quantile, so no single blunder can move it. Read it as the error you should expect in the worst 1 pixel in 20 — a more useful promise than a mean for anything that has to be safe rather than accurate on average.

`bias_valid` **bias** px closer to zero is better

The average of (estimate − truth), with its sign kept. Zero means the errors cancel; a consistent offset points at a calibration or convention problem — an off-by-one in the disparity indexing, or a sub-pixel refinement biased toward the cost minimum's left side — rather than at matching quality.

`density` **density** % higher is better

The denominator behind every '..., answered' column. A method that answers 70% of pixels is being graded on a self-selected 70%. Always read it next to accuracy: a consistency check that raises confidence by discarding ambiguous pixels improves every 'answered' metric and worsens every honest one.

Cost — what did it take?

`wall_s` **wall** s lower is better

Clock time for one pair, median over the configured repeats. It is what latency means, and it is also the noisiest number in the report: a laptop that has warmed up and dropped its clocks drifts by more than the differences being measured. Compare within a run, never across runs.

`cpu_s` **CPU** s lower is better

Total processor time, summed over every core. This is the figure proportional to energy and to what you are renting. Two methods with the same wall time and a 20× gap in CPU time are not equally cheap.

`parallelism` **cores busy** × neither good nor bad

1.0 means single-threaded; 16 means sixteen cores were busy throughout. A single-threaded method's wall time understates how much cheaper it is than a method of the same speed spreading across the machine — and overstates how it will behave once something else wants those cores.

`mdps` **Mdisp/s** higher is better

Throughput normalised by both image size and search range, so it compares a small image with a wide disparity range against a large image with a narrow one. The honest way to compare speed across scenes of different sizes.

`mpix_per_s` **Mpix/s** higher is better

Throughput normalised by image size only. Easier to reason about than Mdisp/s, but it flatters a method run on a scene with a narrow disparity range.

`rss_delta_mb` **RSS growth** MB lower is better

The memory attributable to this method: resident set size at the end minus at the start, sampled on a background thread. An under-estimate for a method reusing memory the allocator already holds, so compare within a family first.

`peak_rss_mb` **process peak** MB neither good nor bad

Carries everything earlier cells in the same process allocated, so it is an upper bound on the process, not a measurement of the algorithm. Re-run one method alone if you need a clean number.

The families

- **learned** — a pretrained deep network, run through ONNX Runtime (marker: diamond)
- **opencv** — OpenCV's own matchers, C++ and SIMD-optimised (marker: circle)
- **scratch** — written here in NumPy, to be read rather than to be fast (marker: square)
- **torch** — the same maths as `scratch`, re-expressed as tensor operations so it can move to a GPU (marker: triangle)

04 Leaderboard

Every algorithm, averaged over the 15 scenes it completed. Click any header to sort; hover one for what the column means.

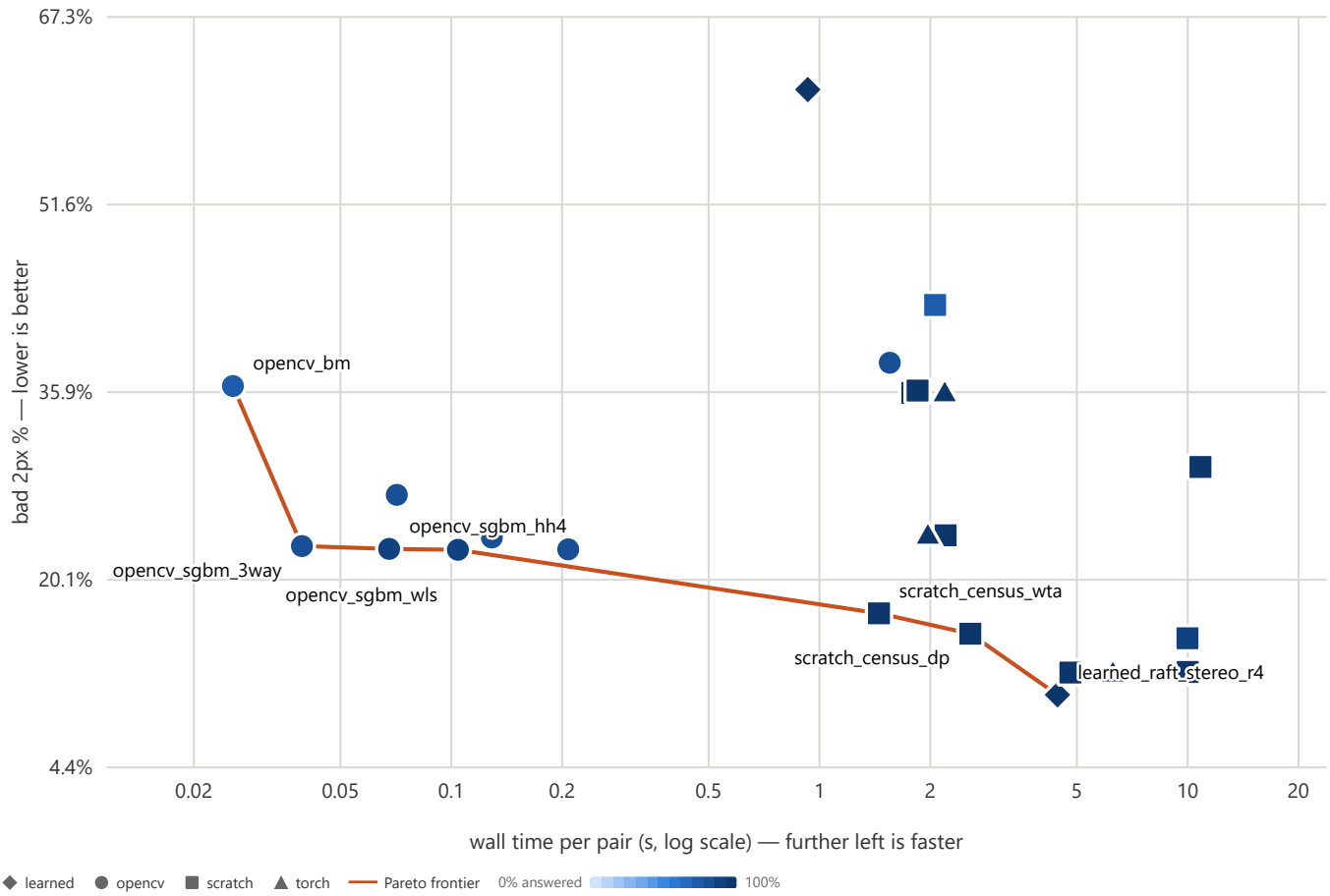
	algorithm	family	bad 2px %	bad 2px, answered %	bad 1px %	bad 0.5px %	EPE px	A95 px	bias px	density %	wall s	CPU s	cores busy ×	Mdisp/s
1	`learned_raft_stereo_r4` FRONTIER	learned	10.5%	10.5%	18.0%	29.1%	1.14	5.46	-0.08	100%	4.43	62.02	14.4×	6.18
2	`torch_census_sgm4` census + semi-global, 4 paths	torch	12.3%	12.3%	17.3%	26.1%	1.91	11.27	+0.81	100%	6.26	71.11	11.9×	4.43
3	`scratch_census_sgm4` census + semi-global, 4 paths	scratch	12.4%	12.4%	17.3%	26.1%	1.91	11.28	+0.81	100%	4.80	4.92	1.0×	5.88
4	`scratch_census_sgm8` census + semi-global, 8 paths	scratch	12.4%	12.4%	17.3%	25.9%	1.96	11.68	+0.74	100%	10.01	9.59	1.0×	2.94
5	`scratch_census_sgm8_lrcheck` census + semi-global, 8 paths + left-right consistency check	scratch	15.2%	8.6%	19.3%	27.2%	1.28	7.12	+0.53	92%	9.99	10.05	1.0×	2.73
6	`scratch_census_dp` FRONTIER census + scanline dynamic programming	scratch	15.6%	15.6%	23.2%	39.5%	2.64	16.07	+0.73	100%	2.57	2.77	1.1×	10.31
7	`scratch_census_wta` FRONTIER census + winner-takes-all	scratch	17.3%	17.3%	22.8%	33.2%	3.06	18.17	+0.26	100%	1.45	1.43	1.0×	19.04
8	`opencv_sgbm_wls` FRONTIER semi-global block matching + WLS edge-aware post-filter	opencv	22.7%	13.6%	27.8%	37.4%	2.18	12.14	+0.26	89%	0.104	0.360	3.6×	266.98
9	`opencv_sgbm_hh` semi-global block matching + full-image dynamic programming	opencv	22.7%	11.6%	26.7%	34.2%	1.75	10.28	+0.98	87%	0.208	0.307	1.5×	130.23
10	`opencv_sgbm_hh4` FRONTIER semi-global block matching + full-image DP, 4 paths	opencv	22.7%	12.0%	26.5%	33.9%	1.81	10.83	+1.04	88%	0.068	0.264	4.2×	421.16
11	`opencv_sgbm_3way` FRONTIER semi-global block matching + 3-way optimisation	opencv	23.0%	12.1%	27.1%	34.8%	1.90	11.37	+1.08	87%	0.039	0.138	3.8×	704.16
12	`opencv_sgbm` semi-global block matching	opencv	23.7%	12.7%	27.9%	35.3%	1.91	11.22	+0.84	87%	0.129	0.184	1.4×	207.27
13	`scratch_zncc_wta` ZNCC + winner-takes-all	scratch	23.9%	23.9%	30.3%	40.4%	4.73	28.72	+1.51	100%	2.20	2.16	1.0×	14.81
14	`torch_zncc_wta` ZNCC + winner-takes-all	torch	23.9%	23.9%	30.3%	40.5%	4.73	28.71	+1.50	100%	1.97	23.45	12.5×	14.42
15	`opencv_bm_wls` block matching + WLS edge-aware post-filter	opencv	27.3%	16.7%	36.6%	49.4%	2.36	12.30	-0.02	87%	0.071	0.287	4.5×	438.08
16	`scratch_sad_sgm8` SAD + semi-global, 8 paths	scratch	29.6%	29.6%	39.5%	53.0%	4.24	19.14	+1.04	100%	10.83	10.88	1.0×	2.53
17	`scratch_sad_wta` SAD + winner-takes-all	scratch	35.8%	35.8%	46.2%	59.9%	5.72	26.13	+0.69	100%	1.79	1.79	1.0×	15.96
18	`torch_sad_wta` SAD + winner-takes-all	torch	35.8%	35.8%	46.2%	60.0%	5.72	26.14	+0.69	100%	2.19	29.57	13.5×	12.33

	algorithm	family	bad 2px %	bad 2px, answered %	bad 1px %	bad 0.5px %	EPE px	A95 px	bias px	density %	wall s	CPU s	cores busy ×	Mdisp/s
19	`scratch_ssd_wta` SSD + winner-takes-all	scratch	36.0%	36.0%	45.9%	59.3%	5.77	26.72	+0.62	100%	1.85	1.86	1.0×	15.95
20	`opencv_bm` FRONTIER block matching	opencv	36.4%	10.5%	39.3%	45.7%	1.93	10.93	+0.26	71%	0.026	0.052	2.0×	1159.75
21	`opencv_quasi_dense`	opencv	38.3%	28.3%	46.0%	58.0%	5.15	20.72	+0.23	86%	1.55	2.22	1.5×	18.79
22	`scratch_sad_wta_lrcheck` SAD + winner-takes-all + left-right consistency check	scratch	43.2%	28.7%	52.5%	65.2%	4.48	23.16	+0.39	78%	2.06	2.09	1.0×	14.33
23	`learned_raft_stereo_r1`	learned	61.2%	61.2%	70.7%	80.8%	25.15	83.02	+21.48	100%	0.930	13.90	15.5×	30.85

05 Accuracy vs time

Lower and further left is better. Marker shape is the family; fill is how much of the image the method answered, which is the third thing you need to read a stereo result honestly.

The trade-off



The frontier is recomputed for whichever metric is on the y axis, so switching to `bad 2px, answered` shows a different — and more flattering — set of winners. That the set changes is the point.

06 What moved the numbers

A leaderboard cannot tell you *why* one method beat another, because several things changed at once. These comparisons change one thing and hold the rest fixed.

Changing only the cost function

How two candidate patches are compared. SAD and SSD add up raw intensity differences; ZNCC normalises each window's mean and variance away; census throws the intensities out entirely and keeps only whether each neighbour is brighter or darker than the centre.

Held fixed: the same family, aggregation and post-processing (scratch / wta).





bad 2px, mean over 15 scenes. Spread across this axis: **18.7 percentage points**.

Changing only the aggregation

What happens to the cost volume before a winner is picked. Winner-takes-all picks per pixel independently. Scanline DP solves each row optimally but ties nothing between rows. Semi-global matching accumulates along 4 or 8 directions, so a pixel's choice is informed by the whole image.

Held fixed: the same family, cost function and post-processing (opencv).



bad 2px, mean over 15 scenes. Spread across this axis: **12.7 percentage points**.

Changing only the backend

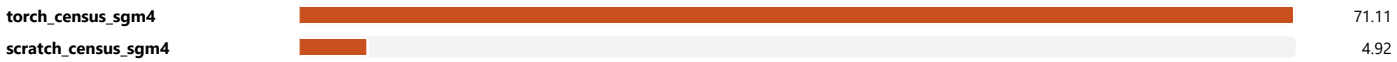
The same arithmetic, executed differently. These should agree on the answer to within floating-point noise; where they differ in accuracy, one of them has a bug. What they do differ in is how much of the machine they use to get there.

Held fixed: the same cost function, aggregation and post-processing (census / sgm4).



bad 2px, mean over 15 scenes. Spread across this axis: **0.0 percentage points**.

The same answer at very different cost



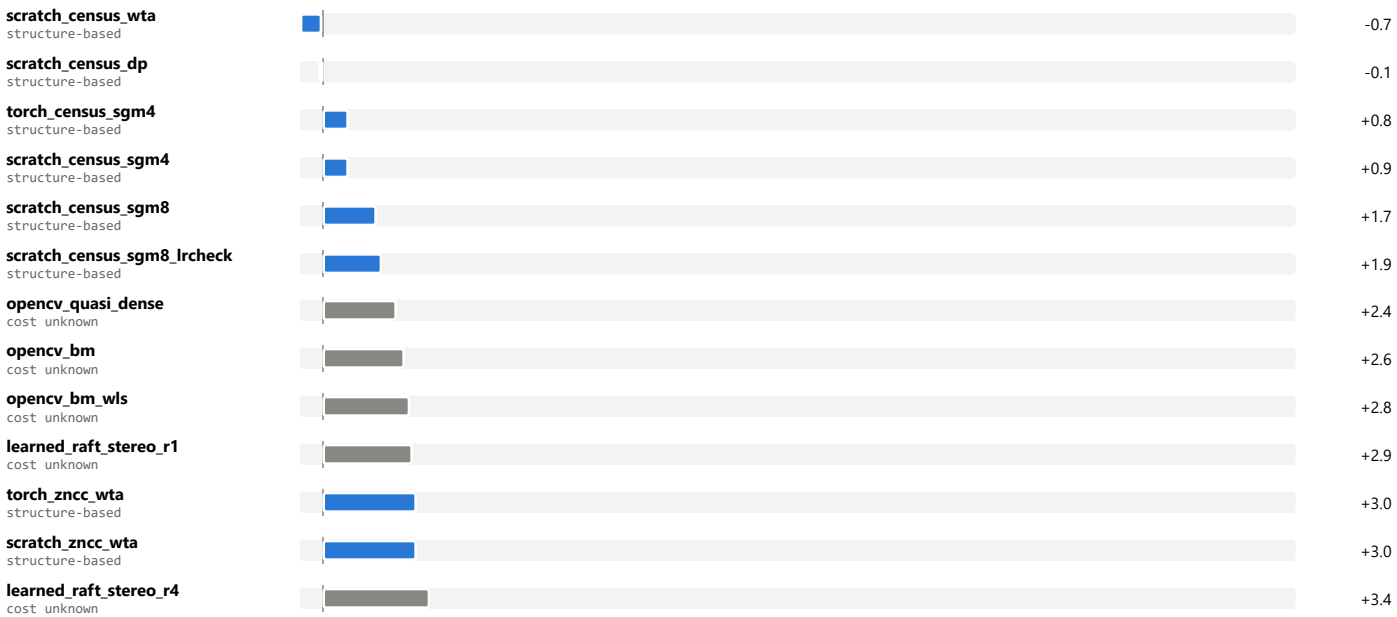
CPU-seconds, summed over every core. Wall time is nearly identical; this is not.

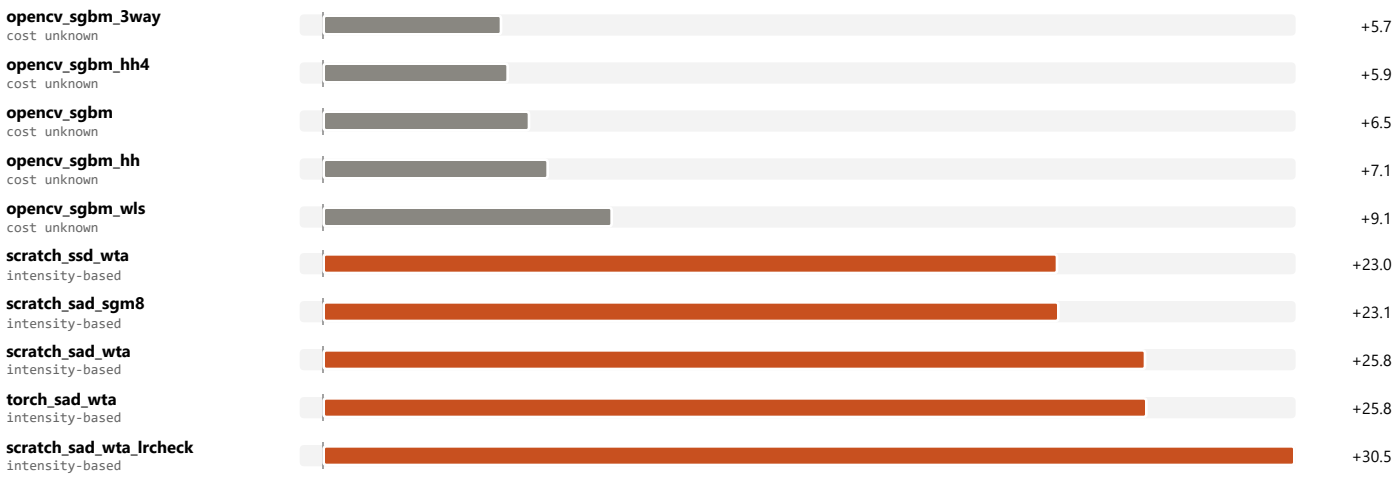
07 Exposure robustness

The cleanest experiment in the run

The dataset ships repeat captures of the same scene that differ only in exposure or lighting. The geometry is identical, so the ground truth is identical, so every extra error a method makes on the variant is caused by the photometric change and nothing else. No other comparison here isolates a single cause this cleanly.

Pairs used: `Motorcycle` → `MotorcycleE`, `Piano` → `PianoL`, `Playtable` → `PlaytableP`.





Change in bad 2px from the base capture to its variant, averaged over the pairs. structure-based cost (census, ZNCC) intensity-based cost (SAD, SSD) cost not identifiable from the registry

08 Scene by scene

An average over scenes hides a method that is merely mediocre everywhere behind one that is excellent except where it collapses.

Every method on every scene

	Adirondack · held	ArtL · held	Jadeplant · held	Motorcycle · tune	MotorcycleE · tune	Piano · tune	PianoL · tune	Pipes · held	Playroom · held	Playtable · held	PlaytableP · held	Recycle · tune	Shelves · tune	Teddy · tune	Vintage · held
learned_raft_stereo_r4	2.8	9.1	32.3	5.8	5.8	7.7	17.4	9.8	9.8	6.6	7.0	4.9	16.5	5.3	16.8
torch_census_sgm4	3.3	12.2	14.7	4.7	4.4	12.1	24.4	9.9	12.2	18.4	8.9	5.1	28.3	5.2	21.3
scratch_census_sgm4	3.3	12.2	14.7	4.7	4.4	12.1	24.5	10.0	12.4	18.4	8.9	5.1	28.3	5.1	21.4
scratch_census_sgm8	3.5	12.2	16.1	4.6	4.3	11.8	24.4	10.4	12.1	16.2	9.0	5.2	28.3	5.7	22.4
scratch_census_sgm8_lrcheck	5.8	15.0	19.8	5.8	5.6	14.5	29.3	12.8	16.2	20.3	11.6	7.3	32.2	6.9	25.6
scratch_census_dp	6.7	13.6	17.7	5.6	5.0	16.7	29.4	12.5	18.1	25.0	12.6	7.4	28.6	6.0	29.3
scratch_census_wta	5.7	15.6	22.1	6.9	6.5	15.4	27.7	14.6	19.0	33.0	19.0	7.8	32.1	8.6	26.0
opencv_sgbm_wls	13.8	24.4	35.8	11.5	19.4	24.3	45.9	14.2	18.3	16.4	14.1	13.0	32.3	14.5	41.9
opencv_sgbm_hh	14.4	23.9	37.6	12.8	16.2	23.5	41.9	16.8	20.0	15.9	15.3	15.3	35.2	16.7	35.0
opencv_sgbm_hh4	14.9	24.3	36.8	12.7	15.2	23.6	40.4	17.0	18.9	17.6	15.9	15.3	36.8	16.2	35.6
opencv_sgbm_3way	15.1	25.2	36.4	12.8	15.8	25.1	42.1	17.5	19.1	19.3	16.3	15.7	32.6	15.2	36.6
opencv_sgbm	15.5	26.3	39.3	13.1	15.7	25.3	42.9	17.8	21.4	18.5	17.8	16.0	31.5	17.1	37.2
scratch_zncc_wta	13.4	28.1	30.7	13.2	12.3	21.1	40.9	21.5	27.2	33.2	23.2	14.3	34.2	12.5	32.5
torch_zncc_wta	13.4	28.2	30.7	13.2	12.3	21.1	40.9	21.5	27.2	33.2	23.2	14.3	34.2	12.5	32.5
opencv_bm_wls	21.5	23.2	48.7	15.4	17.2	20.0	37.7	18.7	26.6	33.3	22.0	21.5	33.7	18.7	50.7
scratch_sad_sgm8	24.1	46.4	35.5	15.3	55.6	29.1	60.9	22.9	24.8	18.8	15.8	14.2	39.0	12.1	29.7
scratch_sad_wta	34.7	47.7	40.3	17.0	60.9	29.2	68.1	27.6	35.9	34.6	29.2	22.2	42.8	17.6	29.3
torch_sad_wta	34.8	47.9	40.3	16.9	60.9	29.2	68.1	27.6	35.9	34.5	29.2	22.2	42.8	17.6	29.3
scratch_ssd_wta	32.0	46.3	43.4	17.5	52.8	30.3	66.1	30.1	35.6	37.1	35.0	22.5	42.7	18.4	30.3
opencv_bm	26.9	36.8	52.9	22.6	23.3	31.5	48.8	27.5	37.2	48.2	38.0	29.3	45.6	25.8	51.6
opencv_quasi_dense	30.7	51.9	72.9	25.3	25.1	31.1	44.8	47.1	38.9	40.7	34.3	25.2	38.4	21.1	47.4
scratch_sad_wta_lrcheck	41.8	55.3	49.4	21.6	75.0	33.0	78.6	33.7	43.5	47.1	39.5	28.1	45.5	20.1	35.6
learned_raft_stereo_r1	47.5	28.5	68.3	54.0	53.1	74.8	84.7	71.1	53.3	84.9	84.4	57.0	63.4	42.8	50.8

better worse (2.8% – 55.3%, clipped at the 95th percentile)

Scene difficulty

The **spread** column is the one to read: it is how much your choice of method matters on that scene. A scene with a small spread is telling you that this decision does not matter there.

scene	size	ndisp	mean	best	by	spread
PianoL TUNE	707×481	65	44.8%	17.4%	learned_raft_stereo_r4	67

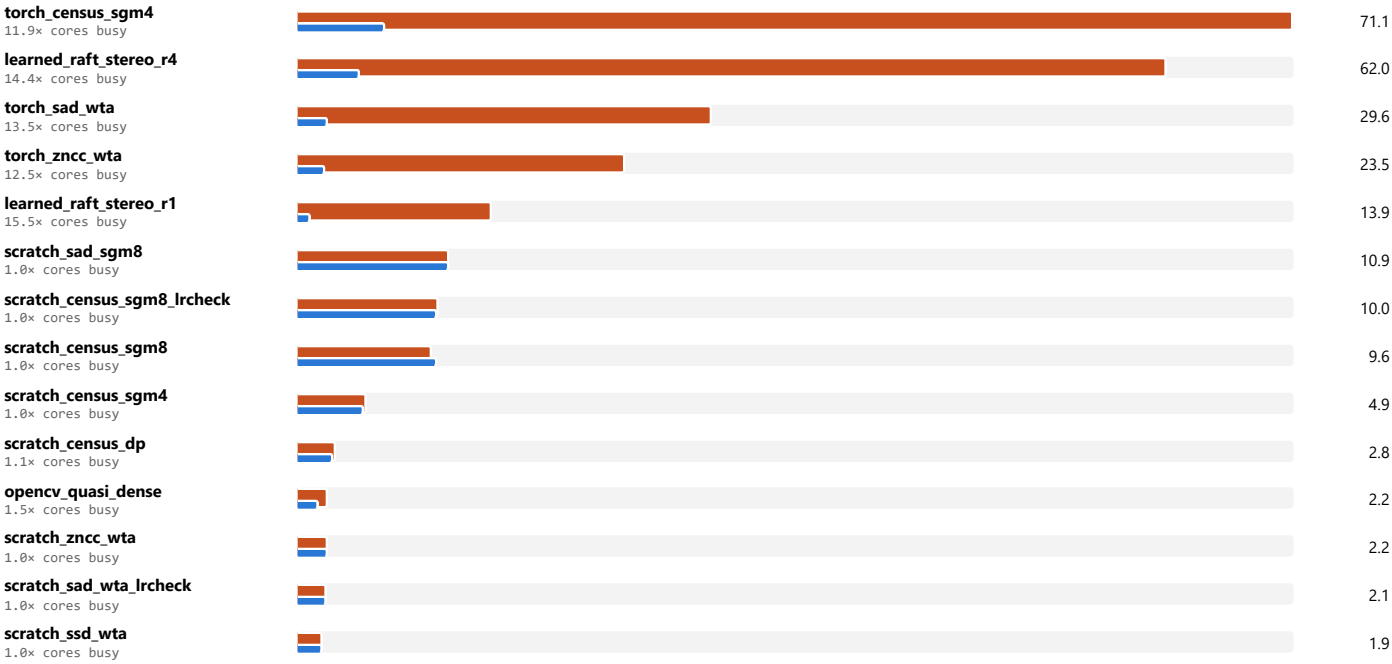
scene	size	ndisp	mean	best	by	spread
Jadeplant HELD-OUT	659×497	160	36.4%	14.7%	`torch_census_sgm4`	58
Shelves TUNE	738×497	60	35.9%	16.5%	`learned_raft_stereo_r4`	47
Vintage HELD-OUT	722×480	190	33.4%	16.8%	`learned_raft_stereo_r4`	35
Playtable HELD-OUT	680×463	73	29.2%	6.6%	`learned_raft_stereo_r4`	78
ArtL HELD-OUT	347×277	64	28.4%	9.1%	`learned_raft_stereo_r4`	46
Playroom HELD-OUT	699×476	83	25.4%	9.8%	`learned_raft_stereo_r4`	44
MotorcycleE TUNE	741×497	70	24.6%	4.3%	`scratch_census_sgm8`	71
Piano TUNE	707×481	65	24.5%	7.7%	`learned_raft_stereo_r4`	67
PlaytableP HELD-OUT	681×462	73	23.1%	7.0%	`learned_raft_stereo_r4`	77
Pipes HELD-OUT	735×485	75	22.3%	9.8%	`learned_raft_stereo_r4`	61
Adirondack HELD-OUT	718×496	73	18.5%	2.8%	`learned_raft_stereo_r4`	45
Recycle TUNE	720×486	65	16.9%	4.9%	`learned_raft_stereo_r4`	52
Teddy TUNE	450×375	64	14.9%	5.1%	`scratch_census_sgm4`	38
Motorcycle TUNE	741×497	70	14.5%	4.6%	`scratch_census_sgm8`	49

09 Compute cost

Three different questions: how long you wait, how much processor you consume, and how much memory it takes.

What each method actually consumes

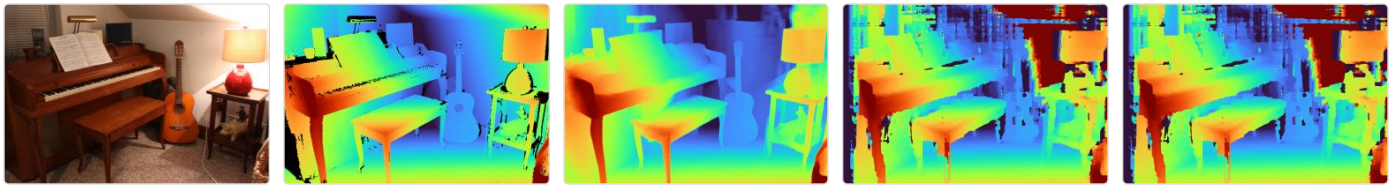
The thin bar is wall time — what you wait. The thick bar is CPU time — what you pay for, summed over every core. Where the thick bar dwarfs the thin one, the method bought its speed by occupying the whole machine, and will not keep that speed when something else wants a core.



10 Looking at the output

Numbers rank; pictures explain. Each method's failure has a characteristic look — horizontal streaks from a row-independent solver, speckle from unaggregated winner-takes-all, black holes from a consistency check that discarded what it could not verify.

PianoL — mean 44.8%, spread 67 points



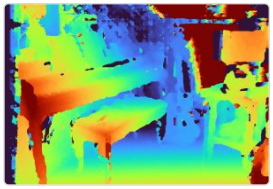
`left image`
the input

`ground truth`
what the answer should be

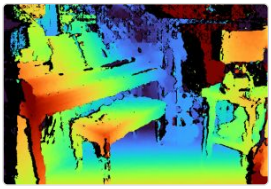
`learned_raft_stereo_r4`
bad 2px 17.4%

`torch_census_sgm4`
bad 2px 24.4%

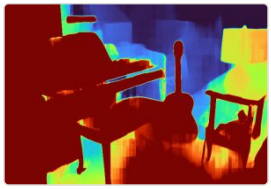
`scratch_census_sgm4`
bad 2px 24.5%



scratch_census_sgm8
bad 2px 24.4%



scratch_census_sgm8_lrcheck
bad 2px 29.3%

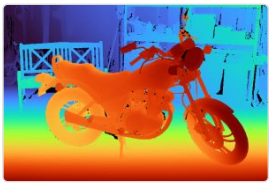


learned_raft_stereo_r1
bad 2px 84.7%

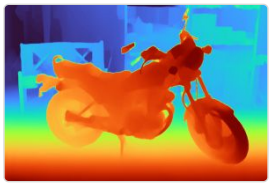
Motorcycle — mean 14.5%, spread 49 points



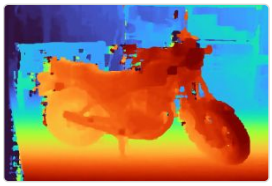
left image
the input



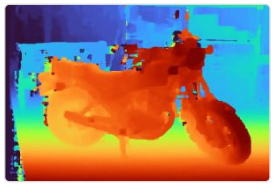
ground truth
what the answer should be



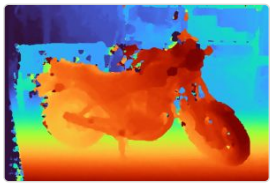
learned_raft_stereo_r4
bad 2px 5.8%



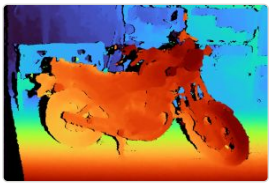
torch_census_sgm4
bad 2px 4.7%



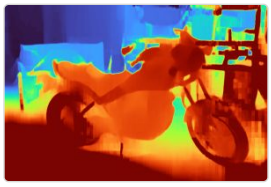
scratch_census_sgm4
bad 2px 4.7%



scratch_census_sgm8
bad 2px 4.6%



scratch_census_sgm8_lrcheck
bad 2px 5.8%



learned_raft_stereo_r1
bad 2px 54.0%

11 What this does not show

The limits of the experiment, stated so a number from it is not quoted further than it can carry.

Timing is only comparable *within* this run

Every cell here was timed as the median of 1 run(s) on one machine in one session. A laptop that has warmed up and dropped its clocks drifts by more than the differences between neighbouring rows, and two runs of this benchmark hours apart have been measured to differ by over 2× on identical code. Ratios inside this table are meaningful; absolute seconds quoted against another run are not. With a single repeat there is no variance estimate at all — re-run with `--repeats 5` before quoting any individual number.

Memory is an under-estimate

`RSS growth` is sampled on a background thread, so a short-lived peak between samples is missed, and a method reusing memory the allocator already holds appears to allocate nothing. `process peak` carries everything earlier cells allocated and is not this method's footprint. Compare within a family first.

Every pair here is already rectified

Matching only ever searches along a row. Rectification — and the calibration behind it — is assumed perfect and is not measured, yet it is where a real system's error usually comes from. A method that wins here can still lose on a rig whose epipolar lines are a pixel off.

Indoor scenes, one dataset

These are controlled Middlebury captures: static, textured, evenly lit, indoors. Nothing here predicts behaviour on a wet road at night, on a specular surface, or on anything moving. The *ordering* of methods is reasonably stable across datasets; the absolute numbers are not.

Defaults were tuned, and tuning can overfit

Per-algorithm defaults in this project came from a grid search restricted to a tune split, with the remaining scenes held out. This run scores both splits together, which is

the honest headline number but mixes scenes the parameters saw with scenes they did not. The scene table marks which is which.

A failed method is still a method

Rows that did not converge are kept in the table and flagged. Dropping them would raise every average and make the field look tighter than it is.

12 Appendix Everything needed to reproduce the run.

All 345 cells completed. A failure here would be recorded as a row rather than aborting the run, so an empty list means an empty list.

Run configuration

`repeats`	1
`warmup`	1
`threads`	None
`eval_region`	nonocc
`save_maps`	False
`save_figures`	True
`max_volume_gb`	4.0
`out_dir`	results

Machine

`platform`	Windows-10-10.0.26200-SP0
`python`	3.11.0
`processor`	Intel64 Family 6 Model 183 Stepping 1, GenuineIntel
`cpu_count_logical`	32
`cpu_count_physical`	24
`ram_total_gb`	34.03
`cpu_brand`	13th Gen Intel(R) Core(TM) i9-13900HX
`opencv`	4.13.0
`opencv_threads`	32
`numpy`	2.3.0
`torch`	2.11.0+cpu
`torch_cuda`	False
`onnxruntime`	1.31.1
`onnxruntime_providers`	['AzureExecutionProvider', 'CPUExecutionProvider']

Generated from `results/20261105-035941` by `python -m bench explain`. The prose is assembled from this run's own measurements, so re-running the benchmark and regenerating this page will change the sentences, not just the tables. Source data: `results.csv`, `run.json`.